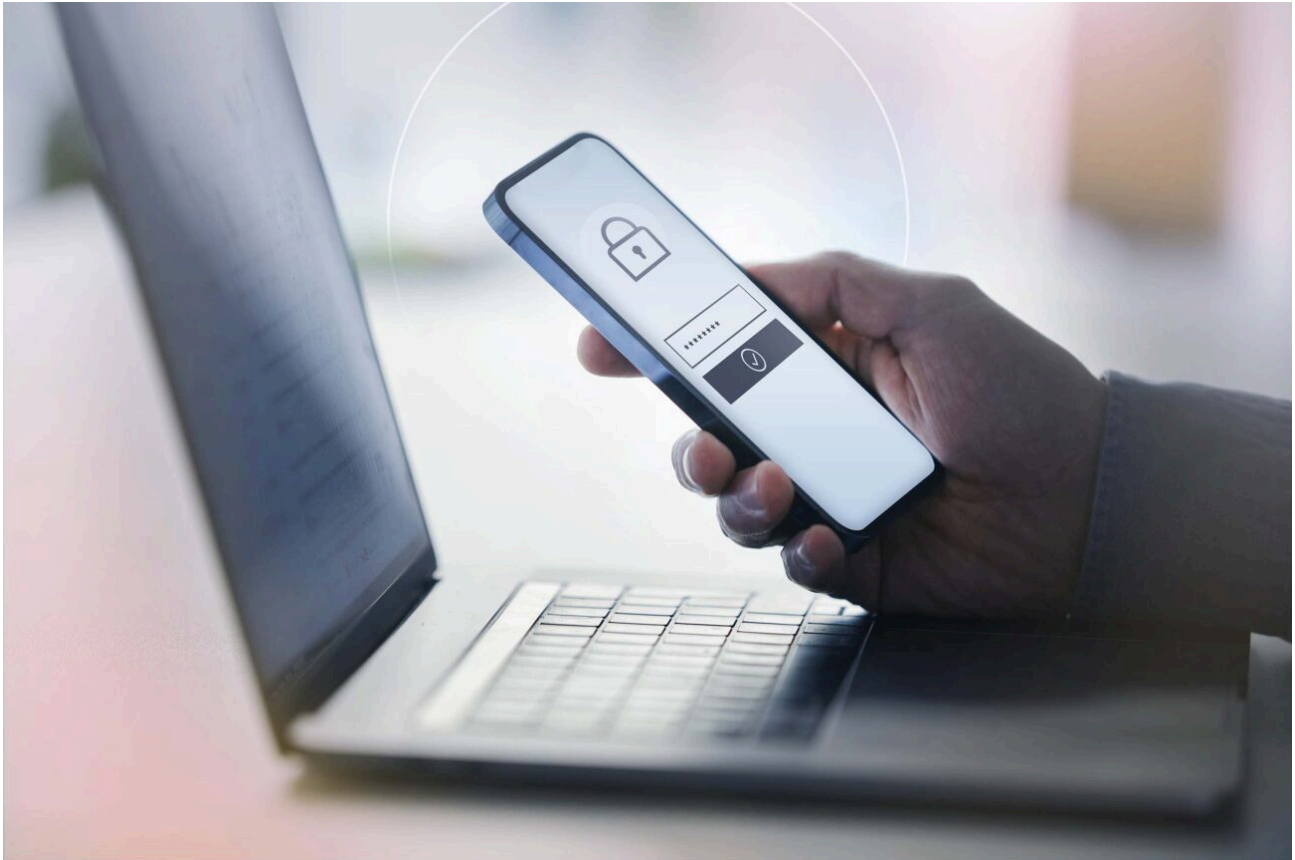


# Bypassing “app-bound” encryption implemented by Google Chrome, in order to steal cookies without administrator rights

 [devoteam.com/expert-view/contournement-du-chiffrement-app-bound-sur-google-chrome-sans-droits-administrateurs](https://devoteam.com/expert-view/contournement-du-chiffrement-app-bound-sur-google-chrome-sans-droits-administrateurs)

March 7, 2025





**Google chrome** is the most used browser in the world, holding around 60% of the market share. This success also marks the success of the project [Chromium](#), which is an open source browser project, very close to Chrome, which serves as a development base for many browsers (Chrome, Microsoft Edge, Opera, Brave, etc.). **Chromium has around 70% market share**. This widespread adoption makes Chromium-based browsers a prime target for attackers.

Indeed, a browser contains a lot of **sensitive data** (cookies, passwords, credit cards, etc.). Criminal groups have therefore developed software called “Infostealers”, which consists of stealing this sensitive data, often without the user’s knowledge.

The **Chrome version 127**, released on July 30, 2024, introduced **App-Bound** encryption on Windows. According to Google, this feature would be designed to protect sensitive browser data against Infostealers, provided that these malicious programs are not run with higher privileges than *chrome.exe*—in other words, not executed with administrator rights.

In this article, we will explore the nuts and bolts of encrypting sensitive data within Chromium, in particular how it is always possible for an attacker to decrypt this data, and this **without administrator rights**.

## **The central functionality of data encryption: DPAPI**

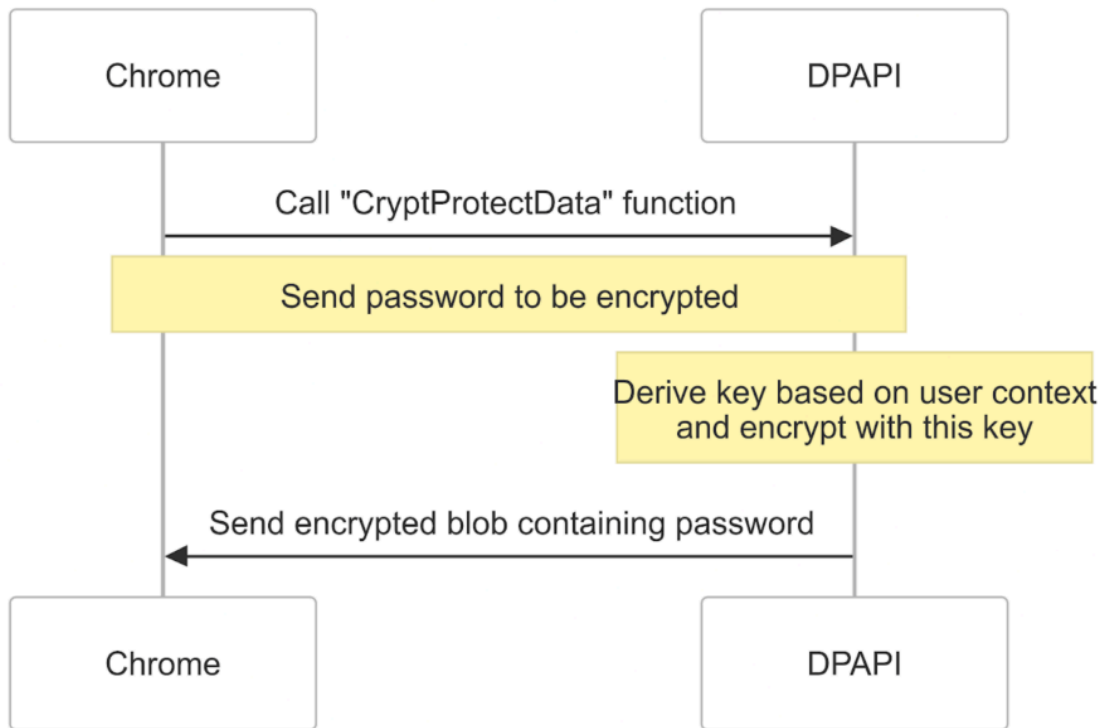
---

Before talking about Chrome’s new encryption method, we must first discuss the main encryption function used previously and which is still partially in use today.

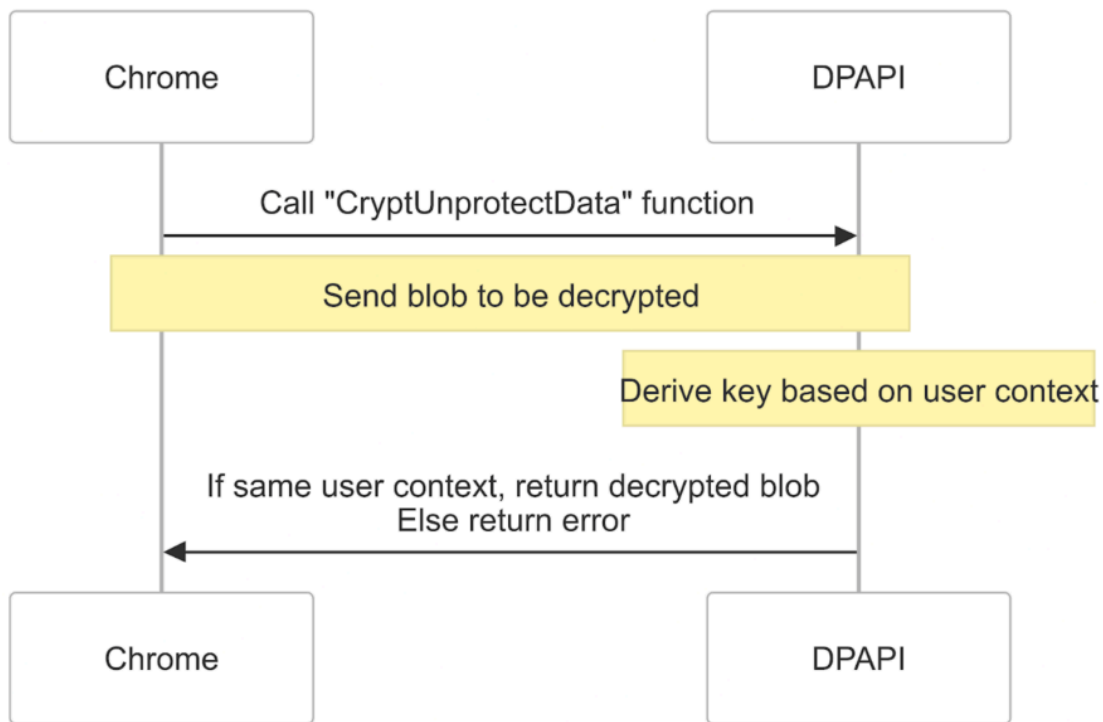
The **DPAPI** (Data Protection API) is a Windows service that helps protect sensitive data. The latter can be used by any developer, and allows the information encryption **using a key derived from the user context**.<sup>2</sup> DPAPI functions are used:

- [CryptProtectData](#) (Encryption),
- [CryptUnprotectData](#) (Decryption).

Here's how they work:



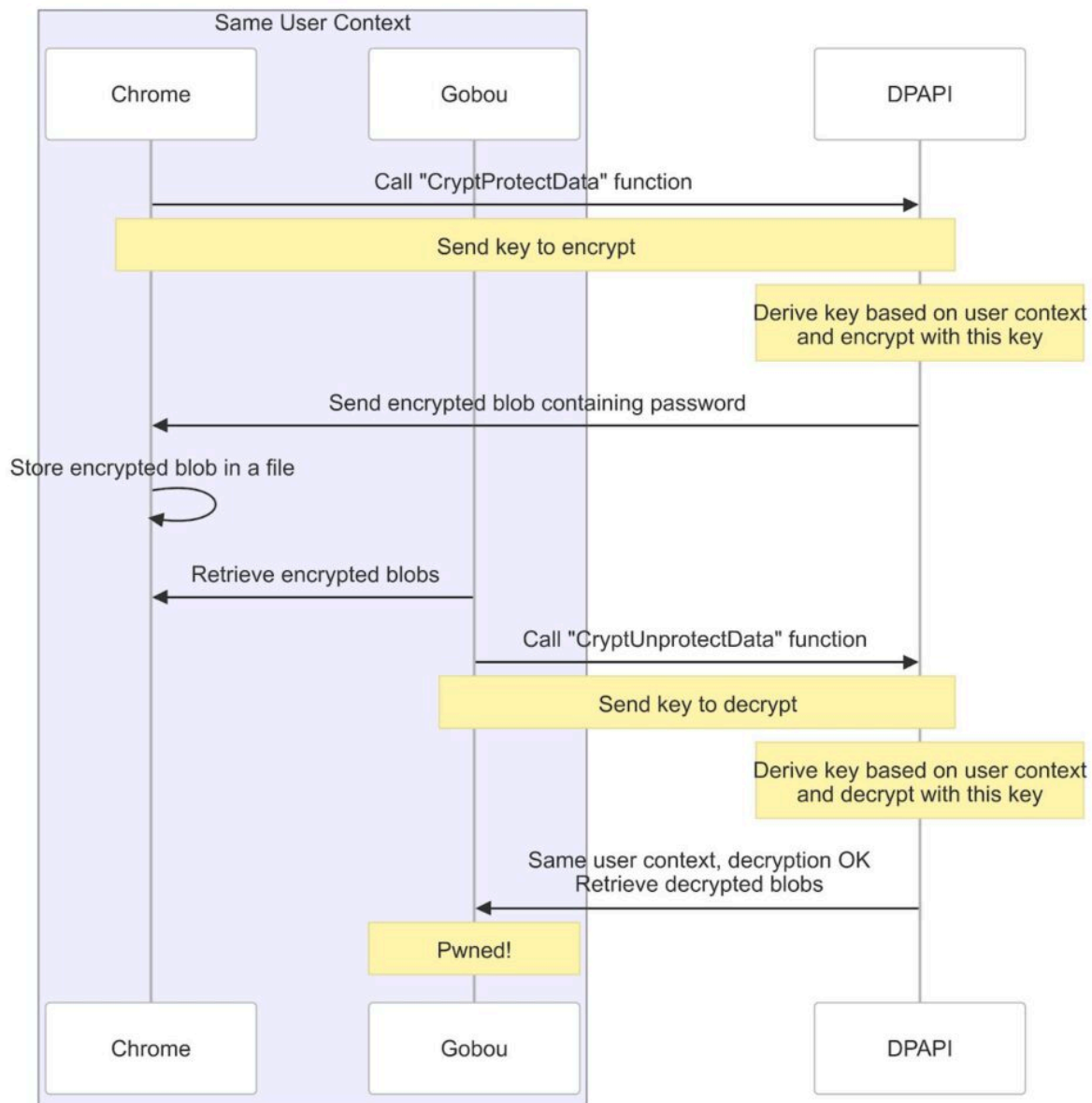
**Diagram corresponding to the function [CryptProtectData](#)**



**Diagramme correspondant à la fonction [CryptUnprotectData](#)**

N.B.: The transfer of the encrypted or plain password is done via the type [DATA\\_BLOB](#) which contains a buffer with the password as well as a pointer to this buffer.

However, the use of DPAPI poses problems. Indeed, a process in the same user context (i.e. without necessarily administrator privileges) **meets all the conditions to be able to decrypt the sensitive data:**



### ***“Weakness” of DPAPI exploited by Infostealers***

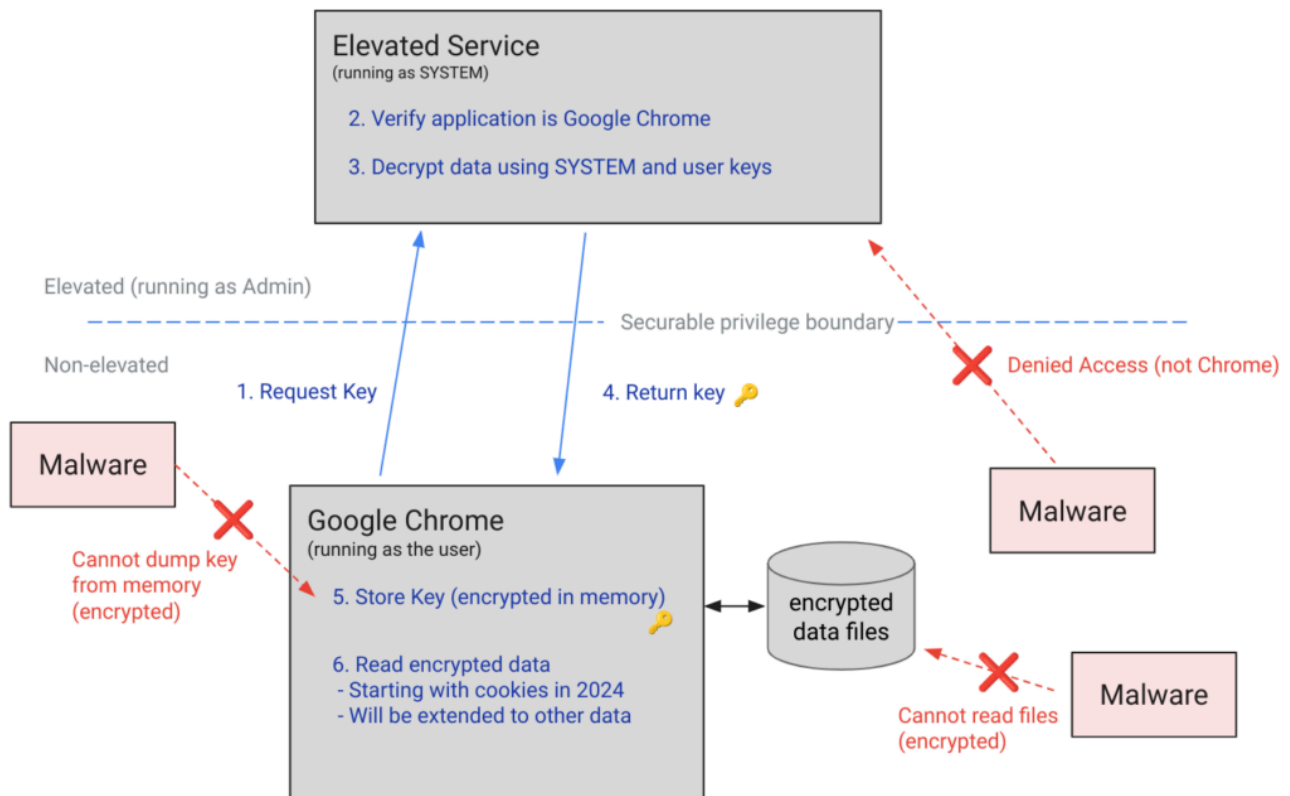
This is how we come to the following conclusion: The **DPAPI** guarantees data protection between Windows users, **but not between processes of the same user**. This weakness was exploited by the attackers, until the implementation of “app-bound” encryption.

## **The introduction of “app-bound” encryption**

On July 30, 2024, in a [post](#) from the “Google security Blog”, Google announces the arrival of encryption “**app-bound**” for cookies in the **version 127**, and for the rest of sensitive data in future versions.

In this post, Google provides a diagram explaining how encryption works **app-bound** :





### Diagram corresponding to app-bound encryption

We note that the call for **DPAPI** is done via a privileged service, **running via the NTAUTHORITY/SYSTEM context**, and not directly by the “chrome.exe” process, which runs in the user context.

Thus, malware running in a user context would no longer be able to decrypt sensitive data using the **DPAPI**, because it operates with lower privileges than the process that encrypted the data.

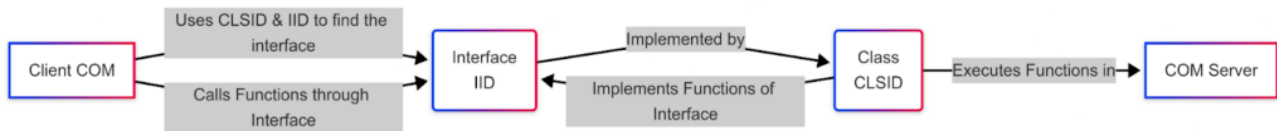
Google announced that the task for Infostealer developers would become more difficult and would now require either:

- A *privilege escalation* carried out by the malware,
- A *injection into Chrome process*, allowing communication with the elevated service (we will come back to this later).

### Workaround via COM (Component Object Model)

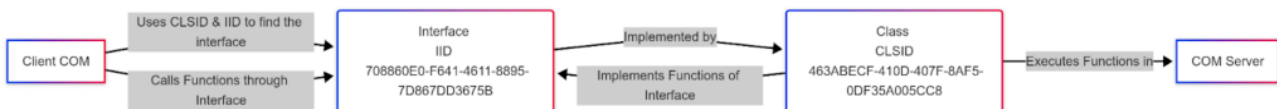
A question remains, how does the Chrome.exe process (user context) communicate with the privileged service (**context SYSTEM**) so that the latter performs his encryption/decryption operations? By analyzing the chrome source code, we notice that it communicates via an object **COM (Component Object Model)**.

This is how a **COM** works:



### ***Diagram corresponding to app-bound encryption***

In the **COM** model, a Class (identified by a **CLSID**) implements one or more interfaces (each identified by an **IID**). By examining the Chromium source code, we can determine the **CLSID** and **IID** fields in advance, since these identifiers are unique, static, [and defined in the Chromium IDL \(Interface Definition Language\) file](#) :



### ***Diagram corresponding to completed app-bound encryption***

In order to identify all CLSID/IIDs for all browsers, the use of [OleViewDotNet](#) allows you to list CLSIDs/IIDs by service, by process, by name, etc. Here are all the CLSID/IIDs by local services:

Registry Properties Interfaces CLSIDs by Name CLSIDs by Local Server **Local Services**

Filter:

- Downloaded Maps Manager
- Enterprise App Management Service
- Enterprise App Management Service
- Enterprise App Management Service
- Enterprise App Management Service
- Extensible Authentication Protocol
- Extensible Authentication Protocol
- Function Discovery Provider Host
- Function Discovery Provider Host
- Function Discovery Provider Host
- Function Discovery Provider Host
- Geolocation Service
- Google Chrome Elevation Service (GoogleChromeElevationService)
  - 708860E0-F641-4611-8895-7D867DD3675B
    - IMarshal
    - Interface {463ABECF-410D-407F-8AF5-0DF35A005CC8}
    - IUnknown
  - Factory Interfaces
- GraphicsPerfSvc
- Hyper-V Heartbeat Service
- Language Experience Service
- Link-Layer Topology Discovery Mapper
- McpManagementService
- Microsoft (R) Diagnostics Hub Standard Collector Service
- Microsoft App-V Client
- Microsoft Edge Elevation Service (MicrosoftEdgeElevationService)
  - 1FCBE96C-1697-43AF-9140-2897C7C69767
    - IMarshal
    - Interface {C9C2B807-7731-4F34-81B7-44FF7779522B}
    - IUnknown
  - Factory Interfaces

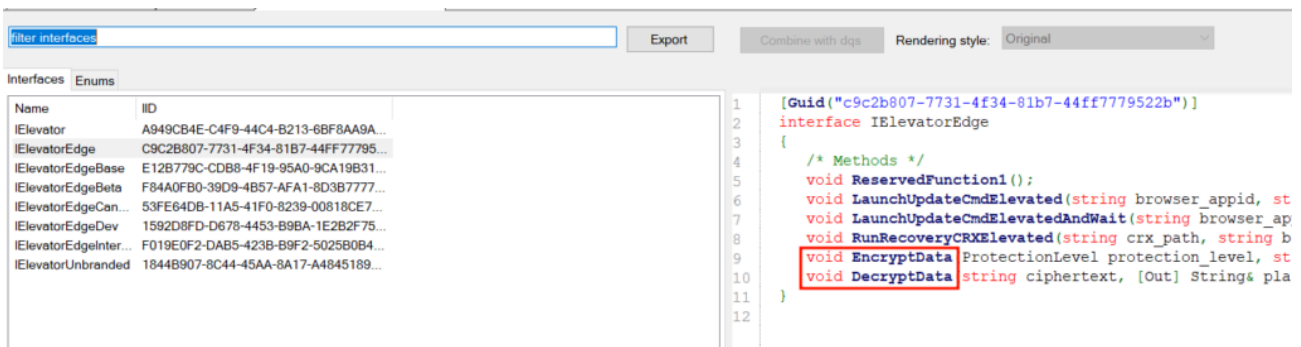
### List of CLSIDs/IIDs by local services on OleViewDotNet

We then note all the CLSID/IID for each browser:

- *Microsoft Edge*
  - CLSID : 1FCBE96C-1697-43AF-9140-2897C7C69767
  - IID : C9C2B807-7731-4F34-81B7-44FF7779522B
- *Chrome*
  - CLSID : 708860E0-F641-4611-8895-7D867DD3675B
  - IID : 463ABECF-410D-407F-8AF5-0DF35A005CC8
- *Brave* :
  - CLSID : 576B31AF-6369-4B6B-8560-E4B203A97A8B
  - IID : F396861E-0C8E-4C71-8256-2FAE6D759CE9



Always with **OleViewDotNet**, it is possible to obtain the functions exposed by these interfaces:



The screenshot shows the OleViewDotNet application. On the left, a list of interfaces is displayed under the 'Interfaces' tab. The 'IEdge' interface is selected. On the right, the C++ code for the 'IEdge' interface is shown. The methods listed are: 'ReservedFunction1()', 'LaunchUpdateCmdElevated', 'LaunchUpdateCmdElevatedAndWait', 'RunRecoveryCRXElevated', 'EncryptData', and 'DecryptData'. The 'EncryptData' and 'DecryptData' methods are highlighted with a red box.

| Name                  | IID                                  |
|-----------------------|--------------------------------------|
| IElevator             | A949CB4E-C4F9-44C4-B213-6BF8AA9A...  |
| IElevatorEdge         | C9C2B807-7731-4F34-81B7-44FF77795... |
| IElevatorEdgeBase     | E12B779C-CDB8-4F19-95A0-9CA19B31...  |
| IElevatorEdgeBeta     | F84A0FB0-39D9-4B57-AFA1-8D3B7777...  |
| IElevatorEdgeCan...   | 53FE64DB-11A5-41F0-8239-00818CE7...  |
| IElevatorEdgeDev      | 1592D8FD-D678-4453-B9BA-1E2B2F75...  |
| IElevatorEdgeInter... | F019E0F2-DAB5-423B-B9F2-5025B0B4...  |
| IElevatorUnbranded    | 1844B907-8C44-45AA-8A17-A4845189...  |

```
[Guid("c9c2b807-7731-4f34-81b7-44ff7779522b")]
interface IElevatorEdge
{
    /* Methods */
    void ReservedFunction1();
    void LaunchUpdateCmdElevated(string browser_appid, st
    void LaunchUpdateCmdElevatedAndWait(string browser_ap
    void RunRecoveryCRXElevated(string crx_path, string b
    void EncryptData ProtectionLevel protection_level, st
    void DecryptData string ciphertext, [Out] String& pla
}
```

### List of functions exposed by the interface dedicated to Edge

You can find the functions *EncryptData* And *DecryptData* that interest us. *OleViewDotNet* also gives the argument types, which now allows us to have all the information to call these functions.

We can proceed to call the “DecryptData” function, but this triggers the following error:



The screenshot shows the Windows Event Viewer. The 'Event 257, Brave-Browser' is selected. The 'Details' tab is active, showing the following error message: '[0101/154506.807:WARNING:caller\_validation.cc(154)] Failed to authenticate caller process: Z:\Elev8\x64\Debug\Elev8.exe'.

### Error log issued by elevated service

A check is therefore carried out when the *DecryptData* function is called.. The latter is found in the “caller\_validation.cc” file, around line 154. We therefore refer [to chromium source code](#) to obtain and understand this validation.

```

namespace {

// Paths look like this: "C:\Program Files\Blah\app.exe".
// This function will remove the final EXE, then it will remove paths that match
// 'Temp', 'Application' or a version pattern if they are the final directory.
//
// Examples:
// "C:\Program Files\Blah\app.exe" ->
// "C:\Program Files\Blah"
//
// "C:\Program Files\Blah\app2.exe" ->
// "C:\Program Files\Blah"
//
// "C:\Program Files\Blah\Temp\app.exe" ->
// "C:\Program Files\Blah"
//
// "C:\Program Files\Blah\Application\app.exe" ->
// "C:\Program Files\Blah"
//
// "C:\Program Files (x86)\Blah\Application\app.exe" ->
// "C:\Program Files\Blah"
//
// "C:\Program Files (x86)\Blah\Application\1.2.3.4\app.exe" ->
// "C:\Program Files\Blah"
//
base::FilePath MaybeTrimProcessPath(const base::FilePath& full_path) {
    auto components = full_path.GetComponents();
    std::vector<std::wstring> trimmed_components;

```

### ***Processing the PATH of the calling executable***

The call to the 'DecryptData' function is intercepted by the privileged service, which analyzes the path of the process calling the function in order to extract the path of the process originating the request. If the result obtained is not "C:\Program Files\Google\Chrome" then an **event 257** is created, and the data decrypted by DecryptData is not returned.

As the "C:\Program Files\" or "C:\Program Files(x86)\\" folder is not writable for users who do not have administrator rights, it is impossible to deploy infostealer directly in this folder to bypass this protection.

## COM Hijack for the win

We therefore have the option of injecting the Chrome.exe process so that it makes the COM calls for us. However, the task seems arduous, particularly given that **the immense majority of known methods for process injections are detected by EDRs** (Endpoint Detection Response).

By analyzing Chrome's behavior, we notice that many COM objects are called by Chrome:

|            |       |            |                                                                   |
|------------|-------|------------|-------------------------------------------------------------------|
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{2E1271D5-2FF2-4EA4-9647-C67A82A2D85C}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{2E1271D5-2FF2-4EA4-9647-C67A82A2D85C}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocServer32  |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocHandler32 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocHandler   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocServer32  |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocHandler32 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\InprocHandler   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\LocalServer32   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\LocalServer     |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\Elevation       |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{144C71F2-7F10-4AB2-BB07-C38F5B9AE05E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocServer32  |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocHandler32 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocHandler   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocServer32  |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocHandler32 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\InprocHandler   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\LocalServer32   |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\LocalServer     |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\Elevation       |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{BCDE0395-E52F-467C-8E3D-C4579291692E}\TreatAs         |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{E77CC89B-7401-4C04-8CED-149DB35ADD04}                 |
| chrome.exe | 12824 | RegOpenKey | HKCR\CLSID\{708860E0-F641-4611-8895-7D867DD3675B}                 |

### Interactions between chrome.exe and COMs detected by Procmon

When a developer needs to create a COM object, they must first declare it. This declaration involves creating CLSIDs using registry keys in the following hives:

- *HKCU\Software\Classes\CLSID* -> For a COM object “user”
- *HKLM\Software\Classes\CLSID* -> For a COM object “system”

N.B.: A user without administrator rights does not have write access to **HKLM (Windows registry information & settings system global)**, but has rights writing to **HKCU** (Windows registry information & settings specific to the logged in user).

Each CLSID contains registry keys whose usefulness is explained below (non-exhaustive list):

- **InProcServer32** : A .DLL or .EXE file which will serve as an entry point for the functions exposed by the COM
- **TreatAs** : Another CLSID that will be used (a proxy in short)
- **ProgID** : A “human readable” name for the CLSID

However, for a given CLSID, Windows will use the following order for loading the COM, only for processes **not privileged** :

1. *HKCU\Software\Classes\CLSID*
2. *HKLM\Software\Classes\CLSID*

HKCU (HKEY\_CURRENT\_USER) can be modified without administrator rights, **therefore we can inject our .DLL via this means.**

Furthermore, during a Google Chrome crash, we can identify .DLL files being loaded from the folder %LOCALAPPDATA%, **available for writing for users** (example :

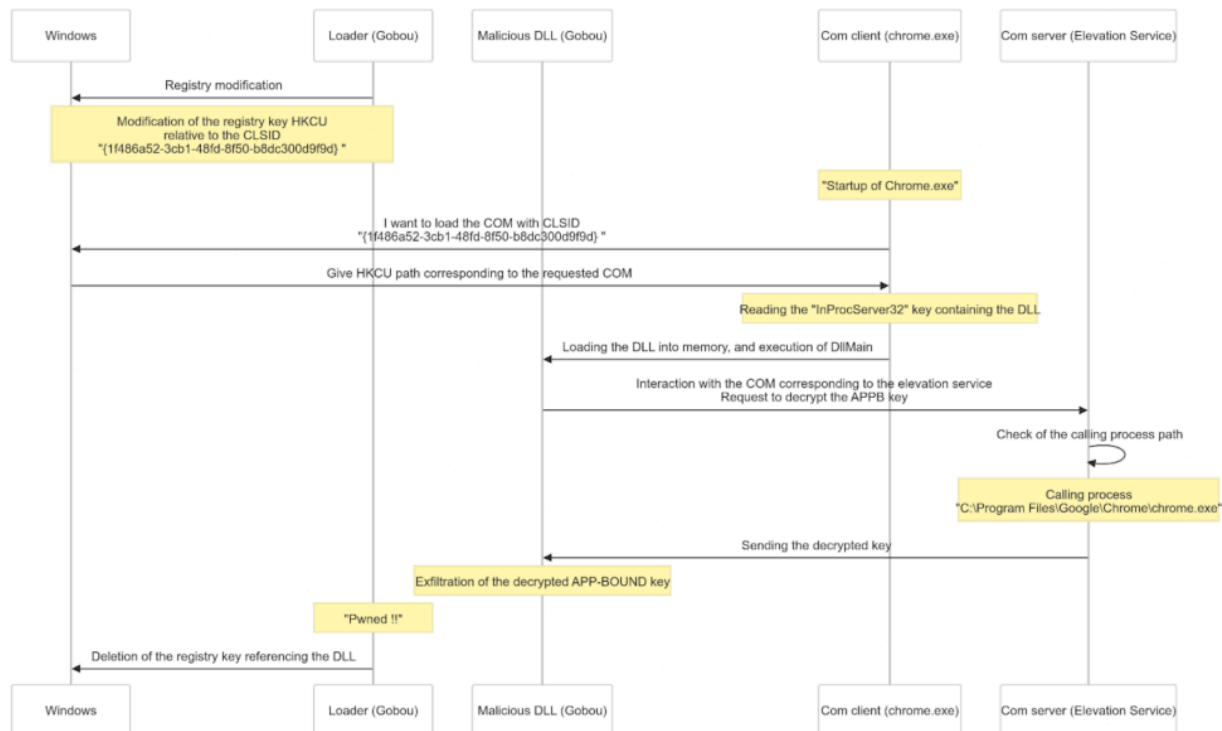
*%LOCALAPPDATA%\Google\Chrome\User*

*Data\MediaFoundationWidevineCdm\x64\1.0.2738.0\Google.Widevine.CDM.dll*”).

However, replacing the DLL of a COM object is not without consequences. **Doing so can disable a potentially important feature** in Chrome and **may cause the process to crash**. After various tests on different CLSIDs, we identified the CLSID “{1f486a52-3cb1-48fd-8f50-b8dc300d9f9d}” **which didn’t cause a crash** when hijacked.

In addition to benefiting from the “legitimacy” of the process **Chrome.exe**, using a COM Hijack **allows for persistence on the system**. It is therefore possible to exfiltrate cookies in real time, as soon as they are recorded, or each time Chrome.exe loads the hijacked COM.

We can therefore establish the following attack path:



## Complete attack path

[A POC allowing you to decrypt this key is available on Github](#), and was developed by "Xaitax". This payload requires administrator rights, since it needs to be placed in the "C:\Program Files\Google\Chrome\" folder. Here is the result of his POC:

```

PS C:\Program Files\Google\Chrome\Application > .\xaitax.exe chrome
-----
Chrome App-Bound Encryption - Decryption
Alexander Hagenah (@xaitax)
-----
[+] Found Chrome Version: 131.0.6778.265
[*] Starting Chrome App-Bound Encryption Decryption process.
[+] COM library initialized.
[+] IElevator instance created successfully.
[+] Proxy blanket set successfully.
[+] Retrieving AppData path.
[+] Local State path: C:\Users\flare_user\AppData\Local\Google\Chrome\User Data\Local State
[+] Base64 encrypted key extracted.
[+] Finished decoding.
[+] Key header is valid.
[+] Encrypted key retrieved: 0100000d08c9ddf0115d1118c7a00c04fc297eb...
[+] BSTR allocated for encrypted key.
[+] Decryption successful.
[+] DECRYPTED KEY: 744d83fba5bf80a2e97cc1b0a2b07985dd6ae319b5d40ef73f46af9c30cb005d
  
```

## Decrypting the key with Xaitax POC

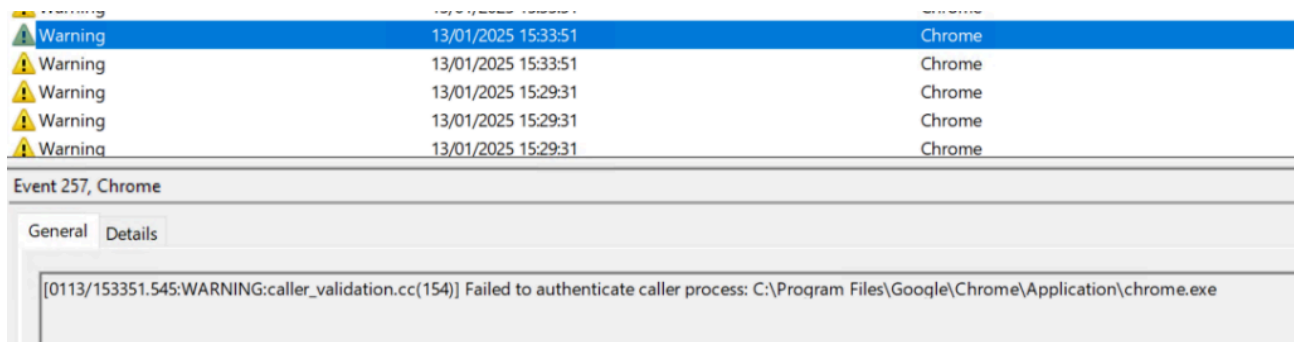
N.B.: After writing this article, Xaitax published another POC which injects itself via 2 techniques in the Chrome process, which doesn't require administrative privileges.

By taking inspiration from the Xaitax POC which allows key recovery, and by coupling it with the COM hijack explained earlier, we arrive at the following result (without administrator rights):









### ***Event 257 created by Google Chrome***

Therefore, here are some CLSIDs that can be used for a COM Hijack:

- Chrome : {1f486a52-3cb1-48fd-8f50-b8dc300d9f9d} (DLL originelle : %SystemRoot%\system32\propsys.dll)
- Brave : {1f486a52-3cb1-48fd-8f50-b8dc300d9f9d} (DLL originelle : %SystemRoot%\system32\propsys.dll)

## **Conclusion**

What are the next steps to secure this sensitive data? The classic recommendation would be to implement **an integrity check of each loaded .DLL** on Chromium, this may however cause side effects for other browsers implementing modified .DLLs.

Furthermore, the implementation of **app-bound** for the rest of the data **imposes a constraint to retain certain functionalities such as importing passwords**, implemented on all known browsers (Edge, Chrome, Brave and Opera). Indeed, the latter would call the COM object of another browser, which is not currently authorized.

Therefore, the implementation of this encryption method for the rest of sensitive data will be closely scrutinized by cybersecurity researchers, notably **as this could open the door to decrypting all data on a single COM object**, as explained in the “To go further” section. N.B.: Special mention for Opera, which always stores its cookies via the classic DPAPI, and which implements its .exe in the folder %APPDATA%.